```
$ docker service ls
ID   NAME                MODE  REP…  IMAGE                  PORTS
ue…  myapp_database      rep…  1/1   postgres:latest
p9…  myapp_db-migrator   rep…  0/1   robisenberg/myapp_w…
ws…  myapp_redis         rep…  1/1   redis:latest
s5…  myapp_web           rep…  3/3   robisenberg/myapp_w…  *:80->3000/tcp
```

So we now have three containers each running a copy of our Rails app. Swarm load balances between replicas of your service, distributing requests to the service across the containers backing that service. However, the load balancing is nondeterministic, and Swarm can send requests to whichever container it sees fit.

Now for the real test: let's see this load balancing in action. Refresh the page http://<instance IP address>/welcome. As you refresh the page several times, you should see the hostname changing, indicating that different containers are handling the request.

You may find that the hostnames don't change when refreshing in a browser; this is likely due to browsers reusing TCP connections across multiple requests. You could try pressing and holding the keyboard shortcut for refresh (Ctrl-R or Cmd-R)—this will reload the page quickly multiple times, which allows you to see the hostname changing.

Alternatively, fetching the page with curl shows you the hostname changing as you make multiple requests:

```
$ curl -4 http://localhost:3000/welcome
```

Note: the -4 here is telling curl to use IPv4.

Swarm gives you the freedom to scale your web app however you'd like by setting the desired number of containers:

```
$ docker service scale myapp_web=<n>
```

Quick Recap

Another action-packed chapter. You've now learned the major tools in our belt for deploying apps with Docker.

Let's review what we covered in this chapter:

1. We introduced Docker Machine and used it to create a virtualized Docker-ready instance:

   ```
   $ docker-machine create --driver virtualbox local-vm-1
   ```

2. We logged onto the new instance using SSH: